AP9EV – Diferenciální evoluce

František Hromek

1. Princip

Diferenciální evoluce (DE) je evoluční algoritmus pro funkce s reálnými parametry. Na rozdíl od GA z minulých úkolů nemusím nic kódovat do bitů – jedinec je rovnou vektor deseti čísel. Pořadí operací je taky jiné: nejdřív mutace, pak křížení a nakonec selekce. Pro každého jedince x_i v populaci se v každé generaci udělá tohle:

– *Mutace* – vezmu rozdíl dvou jiných jedinců, vynásobím ho F a přičtu k bázovému vektoru, tím vznikne mutant v_i. Když se populace stáhne k sobě, rozdíly se zmenší a kroky se zjemní samy.

– *Křížení (binomiální)* – potomek u_i dostane každou souřadnici z mutanta s pravděpodobností CR, jinak z rodiče. Jedna náhodná souřadnice jde z mutanta vždycky.

– *Selekce* – pokud je $f(u_i) \leq f(x_i)$, potomek nahradí rodiče. Populace se tedy nikdy nezhorší.

Jednotlivé verze se liší hlavně mutací:

Verze	**Mutace**	**Nastavení**
Rand/1	$v_i = x_{r1} + F \cdot (x_{r2} - x_{r3})$	F = 0.5, CR = 0.8
Best/1	$v_i = x_{best} + F \cdot (x_{r1} - x_{r2})$	F = 0.5, CR = 0.8
jDE	jako Rand/1, F a CR má každý jedinec svoje	$\tau 1 = \tau 2 = 0.1$, na začátku F = 0.5, CR = 0.9

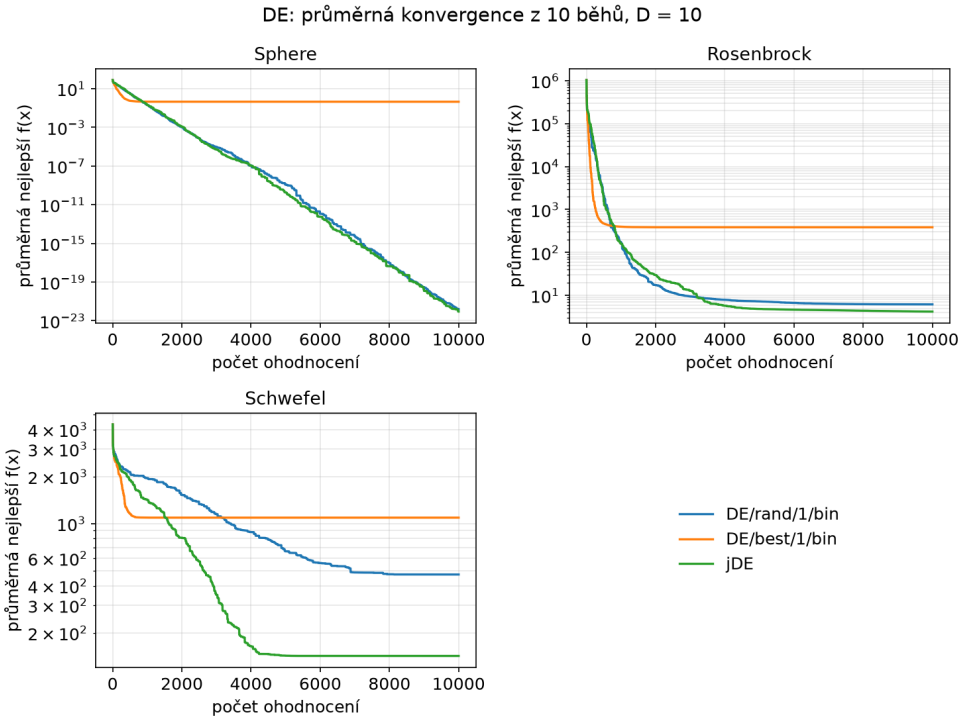
U jDE se před mutací s pravděpodobností 0.1 vylosuje jedinci nové F (z intervalu 0.1 až 0.9) a s pravděpodobností 0.1 nové CR (0 až 1). Když je potomek úspěšný, nové hodnoty si nechá, jinak zůstanou staré. Díky tomu nemusím F a CR ladit ručně.

2. Implementace a experiment

Všechny tři verze jsou v jedné funkci run_de v souboru de.py, jDE se zapíná parametrem jde=True. Testovací funkce Sphere, Rosenbrock a Schwefel jsem převzal z minulého úkolu. Když mutant vyjede mimo meze, souřadnici ořežu na mez.

Nastavení: D = 10, velikost populace NP = 20, 10 000 ohodnocení funkce (500 generací), 10 běhů pro každou kombinaci. Spouští se příkazem python DifEvolution/experiment.py, podrobné statistiky jsou v souboru stats.md. V tabulce je průměrná nejlepší hodnota f na konci běhu:

Verze	**Sphere**	**Rosenbrock**	**Schwefel**
Rand/1	1.5e-22	6.19	472
Best/1	0.444	386	1090
jDE	8.8e-23	4.21	142

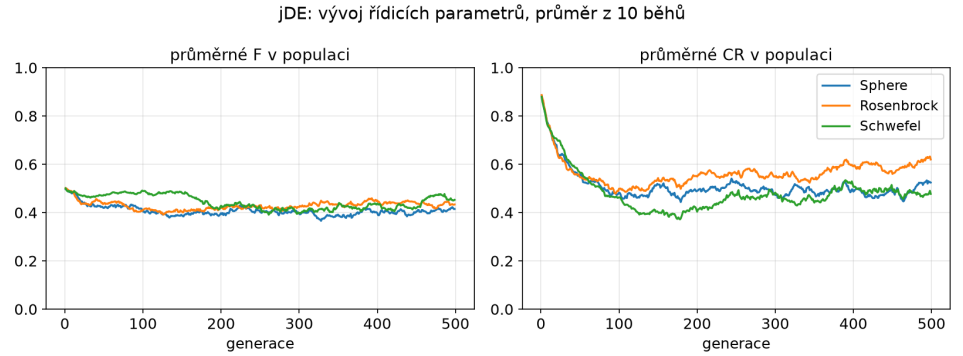


Obrázek 1: Průměrná konvergence z 10 běhů

3. Výsledky

Rand/1 i jDE fungují dobře. Na Sphere klesají rovnoměrně až k 1e-22, na Rosenbrocku se dostanou do údolí a pomalu jdou po jeho dně. Největší rozdíl je na Schwefelovi, kde má jDE průměr 142 a v jednom běhu dokonce našla globální minimum, zatímco Rand/1 skončila na 472.

Best/1 je na začátku nejrychlejší, ale po pár stovkách ohodnocení se zasekne na všech třech funkcích. Nejde o chybu v kódu – protože se všichni mutanti tvoří kolem nejlepšího jedince, celá populace se stáhne do jednoho bodu a rozdíly mezi jedinci jsou pak nulové, takže se algoritmus nemá kam pohnout. Pomůže větší populace: s NP = 50 vyšel Best/1 na Sphere medián 1e-13 místo 0.34.



Obrázek 2: Průměrné F a CR v populaci jDE během běhu

Na obrázku 2 je vidět, že si jDE během prvních zhruba 80 generací sníží CR z 0.9 asi na polovinu a F drží kolem 0.4, a to u všech tří funkcí podobně. Je to tedy níž než pevných 0.8 u Rand/1, takže potomek mění méně souřadnic najednou. Podle přednášky se nižší CR hodí pro separabilní funkce, což by mohlo vysvětlovat, proč je jDE nejvíc lepší právě na Schwefelovi.

4. Závěr

Všechny tři verze DE fungují. Nejlépe vyšla jDE, která má stejnou mutaci jako Rand/1, ale F a CR si nastavuje sama, a byla nejlepší na všech funkcích, nejvíc na Schwefelovi. Rand/1 je spolehlivá a s doporučeným nastavením si vedla dobře, na Rosenbrocku byla lepší než všechny GA z minulého úkolu. Best/1 při malé populaci předčasně zkonverguje a hodí se spíš s větší populací. Hlavně jsem si ověřil, že u DE hodně záleží na nastavení parametrů a jDE tenhle problém jednoduše řeší.